

High-Level Design (HLD): Automation Framework

Project: OrangeHRM Demo Automation

Author: ABHISHEK K M

Version: 1.0

1. Architecture overview

The framework is a layered Page Object Model architecture. The same four layers exist in both the Cypress and Playwright implementations; only the tool-specific plumbing differs.

CI Layer	GitHub Actions: PR smoke, nightly regression, report artifact upload
Test Layer	Spec files – one per module. Contain ONLY: arrange (data), act (page-object calls), assert (expectations). Tagged @smoke / @regression.
Page Object Layer	One class per page/screen. Owns locators + user actions. Returns state; NEVER asserts business outcomes (assertions live in specs).
Support Layer	Fixtures (static test data), data generators (unique names), env config (.env / CI secrets), auth session reuse, custom helpers.

Data flows downward (spec → page object → browser) and **state flows upward** (browser → page object → spec, where assertions happen). Specs never touch a locator; page objects never contain test data or business assertions.

Architecture diagram

```
CI[GitHub Actions: PR smoke / nightly regression] → Runner
Runner[Test Runner: Playwright Test / Cypress] → Specs
Specs[login.spec.js, pim.spec.js ...] → calls actions → Page Objects
Specs → reads → Fixtures & Data Generators
Specs → reads → Env Config (.env / CI secrets)
Page Objects[LoginPage, PIMPage ...] → drives → Browser (Chromium / Firefox / WebKit)
Browser → state → Page Objects → state → Specs
Runner → Reports (HTML Report + Traces/Screenshots)
CI → uploads → Reports
```

2. Design pattern: Page Object Model — and why

Chosen pattern: classic Page Object Model (POM), one class per page, composed where a page has large sub-sections (e.g., PIM's Add Employee form as its own object).

Why POM here specifically:

- **Locator churn is the #1 risk** (Risk R4/R5 in the test plan): OrangeHRM exposes generic `oxd-*` component classes and almost no test IDs. POM confines every locator to exactly one file, so a DOM change after a demo-site upgrade is a one-file fix.
- **Dual-tool parity:** the same page-object interface (class names, method names, signatures) is implemented in both Cypress and Playwright. Specs read almost identically across tools, making the parity requirement (Risk R6) reviewable.
- **Readability as documentation:** specs read as business flows (`loginPage.loginAs(admin) → pimPage.addEmployee(emp)`), which doubles as living documentation for the portfolio.

Rejected alternatives:

- *Raw specs with inline locators* — fastest to write, unmaintainable at 5 modules.
- *Screenplay pattern* — better for very large teams/suites; overhead not justified at this scale.
- *Keyword-driven* — adds an abstraction layer with no consumer (no non-technical test authors here).

3. Folder structure

Target structure (Playwright shown; Cypress mirrors it under its own conventions):

```

playwright/
├─ tests/
│  ├─ login.spec.js
│  ├─ dashboard.spec.js
│  ├─ admin.spec.js
│  ├─ pim.spec.js
│  └─ leave.spec.js
├─ pages/
│  ├─ BasePage.js
│  ├─ LoginPage.js
│  ├─ DashboardPage.js
│  ├─ AdminPage.js
│  └─ pim/
│     ├─ PIMListPage.js
│     └─ AddEmployeePage.js
├─ LeavePage.js
├─ fixtures/testData.js
├─ utils/dataGenerator.js, env.js
├─ playwright.config.js
└─ .github/workflows/smoke.yml, nightly-regression.yml

cypress/
├─ e2e/
│  ├─ login.cy.js
│  ├─ dashboard.cy.js
│  ├─ admin.cy.js
│  ├─ pim.cy.js
│  └─ leave.cy.js
├─ pages/
│  └─ (same class names/API)
├─ fixtures/
│  └─ testData.json
├─ support/
│  ├─ commands.js
│  └─ e2e.js
└─ cypress.config.js

```

Conventions: one spec file per module; one page-object class per page; sub-pages nested in a module folder when a module has more than one significant screen (PIM). BasePage holds only genuinely shared behaviour (navigation, top-bar/menu interactions, toast reading) — not a dumping ground. No spec imports another spec; no page object imports a spec.

4. Tooling choices and rationale

CONCERN	CHOICE	WHY
Runner #1	Playwright Test (JS)	Auto-waiting web-first assertions suit the async oxd-* UI; trace viewer makes CI failures diagnosable without rerunning; free parallelism; three engines from one API
Runner #2	Cypress (JS)	Deliberate second implementation: widest industry adoption; its interactive runner excels during authoring/debugging
Language	JavaScript	Project requirement; one language across both tools keeps fixtures/data rules conceptually shared
Reporting	Playwright HTML report + traces; Mochawesome for Cypress	Both produce self-contained HTML that uploads cleanly as CI artifacts; traces/screenshots attached to failures
CI	GitHub Actions	Repo already lives on GitHub; matrix builds give per-browser jobs; cron for nightly regression; artifacts for reports; secrets for credentials
Lint	ESLint (+ playwright/cypress plugins)	Catches missing awaits, forbidden fixed sleeps, and assertion misuse at review time
Data	JS fixtures + timestamp-based generators	Shared demo tenant (Risks R1/R2) means generated-unique data is mandatory, not optional
Secrets	.env locally, GitHub Actions secrets in CI	Demo creds are public, but the handling pattern must be enterprise-grade

5. Locator strategy (framework-wide rule)

Preference order, applied in every page object and enforced at review:

1. **Role/label/placeholder semantics** — `getByRole`, `getByPlaceholder`, `getByLabel` (Playwright) / equivalent `[name=]`, `:contains` anchoring (Cypress). Survives styling changes.
2. **Stable attributes** — `name`, `type` on form controls (Login form has `name="username"` / `name="password"`).

3. **Anchored text** — menu items and headers by visible text (PIM, Add Employee), scoped to a container.
4. **Scoped oxd-* classes** — last resort, always scoped under a semantic ancestor, never long descendant chains.
5. **Forbidden:** XPath with indexes, nth-child chains, auto-generated attributes, fixed sleeps to "wait for" elements.

6. Execution model

- **Smoke** (@smoke tag / grep): ~8–10 tests, every push/PR, Chromium only, ≤10 min.
- **Regression:** full suite, nightly cron + pre-release, full browser matrix, 1 retry allowed in CI (retried-pass is flagged as flake, not quietly accepted).
- **Auth reuse:** Playwright storageState (login once per worker, reuse session) / Cypress `cy.session()` — login UI itself is only exercised by the Login specs; every other module starts authenticated. This is the single biggest runtime and stability win on a slow shared demo site.